

Métodos Avanzados: medidas de similaridad y distancia - Resolución

Contents

- Ejercicio N°1
- Ejercicio N°2
- Ejercicio N°3
- Ejercicio N°4

Ejercicio N°1

MARIANA vs. MERIANNA

- s_1 : MARIANA
- s_2 : MERIANNA

Las longitudes de estas cadenas son $|s_1| = 7$ y $|s_2| = 8$.

Paso 1: determinar el umbral de coincidencia

Dos caracteres se consideran coincidentes si son iguales y la distancia entre sus posiciones no supera:

$$\left\lceil \frac{\max(|s_1|, |s_2|)}{2} \right\rceil - 1 = 4 - 1 = 3$$

Paso 2: identificar los caracteres coincidentes y calcular m

[Back to top](#)

Alineamos las cadenas posición a posición para facilitar la visualización:

Pos.	1	2	3	4	5	6	7	8
s_1	M	A	R	I	A	N	A	
s_2	M	E	R	I	A	N	N	A

Recorremos s_1 carácter por carácter y buscamos, dentro del umbral de 3 posiciones, si ese carácter aparece en s_2 (sin reutilizar caracteres ya emparejados):

- M (pos. 1): coincide con la M en pos. 1 de s_2 (distancia 0).
- A (pos. 2): coincide con la A en pos. 5 de s_2 (distancia = 3).
- R (pos. 3): coincide con la R en pos. 3 de s_2 (distancia 0).
- I (pos. 4): coincide con la I en pos. 4 de s_2 (distancia 0).
- A (pos. 5): coincide con la A en pos. 8 de s_2 (distancia = 3).
- N (pos. 6): coincide con la N en pos. 6 de s_2 (distancia 0).
- A (pos. 7): no hay ninguna A restante en s_2 .

Por lo tanto, $m = 6$.

Paso 3: identificar las transposiciones y calcular t

Para calcular las transposiciones, construimos una nueva tabla listando únicamente los caracteres coincidentes. Tanto para s_1 como para s_2 los tomamos en el orden en que aparecen en las cadenas originales:

Par	1	2	3	4	5	6
Coincidentes en s_1	M	A	R	I	A	N
Coincidentes en s_2	M	R	I	A	N	A

El primer par coincide en posición. En los últimos cinco, en cambio, el carácter de s_1 y el de s_2 son distintos. Estos son los 5 caracteres transpuestos (resaltados en rojo). Según la definición de Jaro, t es la mitad de ese conteo:

$$t = \frac{5}{2} = 2.5 \approx 2$$

Paso 4: calcular la similaridad de Jaro

Con $m = 6$, $|s_1| = 7$, $|s_2| = 8$ y $t = 2$:

$$sim_J = \frac{1}{3} \left(\frac{6}{7} + \frac{6}{8} + \frac{6-2}{6} \right) = 0.7579$$

Cálculo de la similaridad de Jaro-Winkler

La única coincidencia al inicio de ambas cadenas está dada por la letra M. Con el valor estándar $p = 0.1$:

$$sim_{JW} = 0.7579 + 1 - 0.1(1 - 0.7579) = 0.7821$$

Verificación de los resultados obtenidos:

```
from rapidfuzz.distance import Jaro, JaroWinkler
```

```
# Similaridad de Jaro (redondeada a 4 decimales)
sim_j = round(Jaro.similarity('MARIANA', 'MERIANNA'), 4)
print(sim_j)
```

0.7579

```
# Similaridad de Jaro-Winkler (redondeada a 4 decimales)
sim_jw = round(JaroWinkler.similarity('MARIANA', 'MERIANNA'), 4)
print(sim_jw)
```

0.7821

DELLA CECA vs. DELLACECCA

- s_1 : DELLA CECA
- s_2 : DELLACECCA

Las longitudes de estas cadenas son $|s_1| = 10$ y $|s_2| = 10$.

Paso 1: determinar el umbral de coincidencia

Dos caracteres se consideran coincidentes si son iguales y la distancia entre sus posiciones no supera:

$$\left\lceil \frac{\max(|s_1|, |s_2|)}{2} \right\rceil - 1 = 5 - 1 = 4$$

Paso 2: identificar los caracteres coincidentes y calcular m

Alineamos las cadenas posición a posición para facilitar la visualización:

Pos.	1	2	3	4	5	6	7	8	9	10
s_1	D	E	L	L	A	_	C	E	C	A
s_2	D	E	L	L	A	C	E	C	C	A

Recorremos s_1 carácter por carácter y buscamos, dentro del umbral de 4 posiciones, si ese carácter aparece en s_2 (sin reutilizar caracteres ya emparejados):

- D (pos. 1): coincide con la D en pos. 1 de s_2 (distancia 0).
- E (pos. 2): coincide con la E en pos. 2 de s_2 (distancia 0).
- L (pos. 3): coincide con la R en pos. 3 de s_2 (distancia 0).
- L (pos. 4): coincide con la I en pos. 4 de s_2 (distancia 0).
- A (pos. 5): coincide con la A en pos. 5 de s_2 (distancia 0).
- _ (pos. 6): no hay ningún espacio en s_2 .
- C (pos. 7): coincide con la C en pos. 6 de s_2 (distancia = 1).
- E (pos. 8): coincide con la E en pos. 7 de s_2 (distancia = 1).
- C (pos. 9): coincide con la C en pos. 8 de s_2 (distancia = 1), que está primera y todavía no fue usada.
- A (pos. 10): coincide con la A en pos. 10 de s_2 (distancia 0).

Por lo tanto, $m = 9$.

Paso 3: identificar las transposiciones y calcular t

Para calcular las transposiciones, construimos una nueva tabla listando únicamente los caracteres coincidentes. Tanto para s_1 como para s_2 los tomamos en el orden en que aparecen en las cadenas originales:

Par	1	2	3	4	5	6	7	8	9
Coincidentes en s_1	D	E	L	L	A	C	E	C	A
Coincidentes en s_2	D	E	L	L	A	C	E	C	A

Todos los pares coinciden en posición.

$$t = \frac{0}{2} = 0$$

Paso 4: calcular la similaridad de Jaro

Con $m = 9$, $|s_1| = 10$, $|s_2| = 10$ y $t = 0$:

$$sim_J = \frac{1}{3} \left(\frac{9}{10} + \frac{9}{10} + \frac{9}{9} \right) = 0.9333$$

Cálculo de la similaridad de Jaro-Winkler

Ambas cadenas comparten el prefijo **DELLA** (5 caracteres), pero Jaro-Winkler considera hasta un máximo de 4, por lo que $l = 4$. Con el valor estándar $p = 0.1$:

$$sim_{JW} = 0.9333 + 4 \cdot 0.1(1 - 0.9333) = 0.9600$$

Verificación de los resultados obtenidos:

```
# Similaridad de Jaro (redondeada a 4 decimales)
sim_j = round(Jaro.similarity('DELLA CECA', 'DELLACECCA'), 4)
print(sim_j)
```

0.9333

```
# Similaridad de Jaro-Winkler (redondeada a 4 decimales)
sim_jw = round(JaroWinkler.similarity('DELLA CECA', 'DELLACECCA'), 4)
print(sim_jw)
```

0.96

CÓRDOBA 2568 vs. CORDOBA 2478

- s_1 : **CÓRDOBA 2568**
- s_2 : **CORDOBA 2478**

Las longitudes de estas cadenas son $|s_1| = 12$ y $|s_2| = 12$.

Paso 1: determinar el umbral de coincidencia

Dos caracteres se consideran coincidentes si son iguales y la distancia entre sus posiciones no supera:

$$\left\lceil \frac{\max(|s_1|, |s_2|)}{2} \right\rceil - 1 = 6 - 1 = 5$$

Paso 2: identificar los caracteres coincidentes y calcular m

Alineamos las cadenas posición a posición para facilitar la visualización:

Pos .	1	2	3	4	5	6	7	8	9	10	11	12
s_1	C	Ó	R	D	O	B	A	_	2	5	6	8
s_2	C	O	R	D	O	B	A	_	2	4	7	8

Recorremos s_1 carácter por carácter y buscamos, dentro del umbral de 5 posiciones, si ese carácter aparece en s_2 (sin reutilizar caracteres ya emparejados):

- C (pos. 1): coincide con la C en pos. 1 de s_2 (distancia 0).
- Ó (pos. 2): no hay ninguna Ó (con tilde) en s_2 .
- R (pos. 3): coincide con la R en pos. 3 de s_2 (distancia 0).
- D (pos. 4): coincide con la D en pos. 4 de s_2 (distancia 0).
- O (pos. 5): coincide con la primera O en pos. 2 de s_2 (distancia = 3).
- B (pos. 6): coincide con la B en pos. 6 de s_2 (distancia 0).
- A (pos. 7): coincide con la A en pos. 7 de s_2 (distancia 0).
- _ (pos. 8): coincide con el espacio en pos. 8 de s_2 (distancia 0).
- 2 (pos. 9): coincide con el 2 en pos. 9 de s_2 (distancia 0).
- 5 (pos. 10): no hay ningún 5 en s_2 .
- 6 (pos. 11): no hay ningún 6 en s_2 .
- 8 (pos. 12): coincide con el 8 en pos. 12 de s_2 (distancia 0).

Por lo tanto, $m = 9$.

Paso 3: identificar las transposiciones y calcular t

Para calcular las transposiciones, construimos una nueva tabla listando únicamente los caracteres coincidentes. Tanto para s_1 como para s_2 los tomamos en el orden en que aparecen en las cadenas originales:

Par	1	2	3	4	5	6	7	8	9
Coincidentes en s_1	C	R	D	O	B	A	—	2	8
Coincidentes en s_2	C	O	R	D	B	A	—	2	8

Como se observa, en los pares 2, 3 y 4 el carácter de s_1 y el de s_2 son distintos. Estos son los 3 caracteres transpuestos (resaltados en rojo). Según la definición de Jaro, t es la mitad de ese conteo:

$$t = \frac{3}{2} = 1.5 \approx 1$$

Paso 4: calcular la similaridad de Jaro

Con $m = 9$, $|s_1| = 12$, $|s_2| = 12$ y $t = 1$:

$$sim_J = \frac{1}{3} \left(\frac{9}{12} + \frac{9}{12} + \frac{9-1}{9} \right) = 0.7963$$

Cálculo de la similaridad de Jaro-Winkler

La única coincidencia al inicio de ambas cadenas está dada por la letra C. Con el valor estándar $p = 0.1$:

$$sim_{JW} = 0.7963 + 0.1(1 - 0.7963) = 0.8167$$

Verificación de los resultados obtenidos:

```
# Similaridad de Jaro (redondeada a 4 decimales)
sim_j = round(Jaro.similarity('CÓRDOBA 2568', 'CORDOBA 2478'), 4)
print(sim_j)
```

0.7963

```
# Similaridad de Jaro-Winkler (redondeada a 4 decimales)
sim_jw = round(JaroWinkler.similarity('CÓRDOBA 2568', 'CORDOBA 2478'), 4)
print(sim_jw)
```

0.8167

SAN MARTÍN vs. ASNMARTÍN

- s_1 : SAN MARTÍN
- s_2 : ASNMARTÍN

Las longitudes de estas cadenas son $|s_1| = 10$ y $|s_2| = 9$.

Paso 1: determinar el umbral de coincidencia

Dos caracteres se consideran coincidentes si son iguales y la distancia entre sus posiciones no supera:

$$\left\lceil \frac{\max(|s_1|, |s_2|)}{2} \right\rceil - 1 = 5 - 1 = 4$$

Paso 2: identificar los caracteres coincidentes y calcular m

Alineamos las cadenas posición a posición para facilitar la visualización:

Pos.	1	2	3	4	5	6	7	8	9	10
s_1	S	A	N	_	M	A	R	T	Í	N
s_2	A	S	N	M	A	R	T	Í	N	

Recorremos s_1 carácter por carácter y buscamos, dentro del umbral de 4 posiciones, si ese carácter aparece en s_2 (sin reutilizar caracteres ya emparejados):

- S (pos. 1): coincide con la S en pos. 2 de s_2 (distancia = 1).
- A (pos. 2): coincide con la A en pos. 1 de s_2 (distancia = 1).
- N (pos. 3): coincide con la N en pos. 3 de s_2 (distancia 0).
- _ (pos. 4): no hay ningún espacio () en s_2 .
- M (pos. 5): coincide con la M en pos. 4 de s_2 (distancia = 1).
- A (pos. 6): coincide con la A en pos. 5 de s_2 (distancia = 1).
- R (pos. 7): coincide con la R en pos. 6 de s_2 (distancia = 1).
- T (pos. 8): coincide con la T en pos. 7 de s_2 (distancia = 1).
- Í (pos. 9): coincide con la Í en pos. 8 de s_2 (distancia = 1).
- N (pos. 10): coincide con la N en pos. 9 de s_2 (distancia = 1).

Por lo tanto, $m = 9$.

Paso 3: identificar las transposiciones y calcular t

Para calcular las transposiciones, construimos una nueva tabla listando únicamente los caracteres coincidentes. Tanto para s_1 como para s_2 los tomamos en el orden en que aparecen en las cadenas originales:

Par	1	2	3	4	5	6	7	8	9
Coincidentes en s_1	S	A	N	M	A	R	T	Í	N
Coincidentes en s_2	A	S	N	M	A	R	T	Í	N

Como se observa, en los primeros tres pares el carácter de s_1 y el de s_2 son distintos. Estos son los 3 caracteres transpuestos (resaltados en rojo). Según la definición de Jaro, t es la mitad de ese conteo:

$$t = \frac{3}{2} = 1.5 \approx 1$$

Paso 4: calcular la similaridad de Jaro

Con $m = 9$, $|s_1| = 10$, $|s_2| = 9$ y $t = 1$:

$$sim_J = \frac{1}{3} \left(\frac{9}{10} + \frac{9}{9} + \frac{9-1}{9} \right) = 0.9296$$

Cálculo de la similaridad de Jaro-Winkler

No hay ninguna coincidencia en el inicio de las dos cadenas, motivo por el cual la similaridad de Jaro-Winkler es igual a la de Jaro.

$$sim_{JW} = 0.9296$$

Verificación de los resultados obtenidos:

```
# Similaridad de Jaro (redondeada a 4 decimales)
sim_j = round(Jaro.similarity('SAN MARTÍN', 'ASN MARTÍN'), 4)
print(sim_j)
```

0.9296

```
# Similaridad de Jaro-Winkler (redondeada a 4 decimales)
sim_jw = round(JaroWinkler.similarity('SAN MARTÍN', 'ASN MARTÍN'), 4)
print(sim_jw)
```

0.9296

Ejercicio N°2

```
import pandas as pd

# Base clientes
clientes = pd.read_excel('datasets/clientes_base.xlsx')
clientes.head()
```

	nombre_cliente	ciudad	email	fecha_registro
0	Lucía Fernández	Buenos Aires	lucia.fernandez@email.com	2021-03-15
1	Martín Gómez	Córdoba	martin.gomez@email.com	2020-07-22
2	Valentina Torres	Rosario	valentina.torres@email.com	2022-01-10
3	Sebastián Ruiz	Mendoza	sebastian.ruiz@email.com	2019-11-05
4	Camila Herrera	Tucumán	camila.herrera@email.com	2023-02-28

```
# Base ventas
ventas = pd.read_excel('datasets/ventas.xlsx')
ventas.head()
```

	id_venta	fecha	nombre_cliente	producto	monto_usd
0	1	2024-01-05	Lucía Fernández	iPhone 15	1299.99
1	2	2024-01-12	Martin Gomez	MacBook Air	1899.99
2	3	2024-01-20	Valentina Torres	iPad Pro	999.99
3	4	2024-02-03	Nicolás Vargas	MacBook Pro	2499.99
4	5	2024-02-14	Nicolas Vargas	iPhone 15	1299.99

1. ¿Cuál fue el monto total de venta de productos *iPad* y *MacBook*?

Para responder a esta pregunta, utilizamos el dataset de ventas. Analizamos los productos vendidos:

```
ventas['producto'].unique()
```

```
<StringArray>
[ 'iPhone 15', 'MacBook Air',      'iPad Pro', 'MacBook Pro', 'AirPods Pro',
  'iPad Air', 'Apple Watch']
Length: 7, dtype: str
```

Vemos que hay varios tipos de *iPad* y también de *MacBook*. Para filtrar los registros que corresponden a estos productos, utilizamos el método `str.contains`:

```
ventas_filtrado = ventas[ventas['producto'].str.contains('iPad|MacBook')]
ventas_filtrado
```

	id_venta	fecha	nombre_cliente	producto	monto_usd
1	2	2024-01-12	Martin Gomez	MacBook Air	1899.99
2	3	2024-01-20	Valentina Torres	iPad Pro	999.99
3	4	2024-02-03	Nicolás Vargas	MacBook Pro	2499.99
6	7	2024-03-07	Nicolás Vargas	iPad Air	749.99
8	9	2024-03-22	Sofía Méndez	MacBook Air	1899.99
10	11	2024-04-10	Tomas Aguirre	iPad Pro	999.99
11	12	2024-04-18	Sebastián Ruiz	MacBook Pro	2499.99
13	14	2024-05-15	Nicolás Vargas	MacBook Air	1899.99
16	17	2024-06-20	Florencia Castro	iPad Pro	999.99
17	18	2024-07-08	Agustín Romero	MacBook Pro	2499.99

Calculamos el monto total utilizando el método `sum`:

```
monto_total = ventas_filtrado['monto_usd'].sum()

print(f'El monto total de venta de estos productos fue {monto_total} USD')
```

El monto total de venta de estos productos fue 16949.899999999998 USD

2. Realice la unión de ambos DataFrames con `nombre_cliente` como *key* utilizando el *join* que permita que se conserve todos los registros de las ventas. ¿Cuántos registros del DataFrame resultante presentan valores faltantes en las columnas provenientes de `clientes_base`? ¿A qué atribuye ese resultado?

Hacemos una operación *merge* considerando a la columna `nombre_cliente` como *key* y seteando el parámetro `how = 'left'` para que el DataFrame resultante conserve todos los registros de las ventas:

```
df = pd.merge(ventas, clientes, on = 'nombre_cliente', how = 'left')
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id_venta         20 non-null    int64
1   fecha           20 non-null    str
2   nombre_cliente  20 non-null    str
3   producto        20 non-null    str
4   monto_usd       20 non-null    float64
5   ciudad          14 non-null    str
6   email           14 non-null    str
7   fecha_registro  14 non-null    str
dtypes: float64(1), int64(1), str(6)
memory usage: 1.4 KB
```

Observamos que en cada una de las columnas que provienen del DataFrame `clientes` (`ciudad`, `email` y `fecha_registro`) hay un total de 6 valores faltantes. Exploramos la tabla resultante para encontrar una posible causa:

```
# Ordenamos df según nombre del cliente
df.sort_values('nombre_cliente')
```

	id_venta	fecha	nombre_cliente	producto	monto_usd	ciudad	e
7	8	2024-03-15	Agustin Romero	Apple Watch	499.99	NaN	NaN
17	18	2024-07-08	Agustín Romero	MacBook Pro	2499.99	Córdoba	agustin.romero@email.c
15	16	2024-06-03	Camila Herrera	iPhone 15	1299.99	Tucumán	camila.herrera@email.c
5	6	2024-02-28	Camila Herrera	AirPods Pro	279.99	Tucumán	camila.herrera@email.c
9	10	2024-04-01	Florencia Castro	iPhone 15	1299.99	Buenos Aires	florencia.castro@email.c
16	17	2024-06-20	Florencia Castro	iPad Pro	999.99	Buenos Aires	florencia.castro@email.c
12	13	2024-05-02	Lucía Fernandez	AirPods Pro	279.99	NaN	NaN
0	1	2024-01-05	Lucía Fernández	iPhone 15	1299.99	Buenos Aires	lucia.fernandez@email.c
1	2	2024-01-12	Martin Gomez	MacBook Air	1899.99	NaN	NaN
4	5	2024-02-14	Nicolas Vargas	iPhone 15	1299.99	NaN	NaN
18	19	2024-07-19	Nicolás Vargas	AirPods Pro	279.99	La Plata	nicolas.vargas@email.c
13	14	2024-05-15	Nicolás Vargas	MacBook Air	1899.99	La Plata	nicolas.vargas@email.c
3	4	2024-02-03	Nicolás Vargas	MacBook Pro	2499.99	La Plata	nicolas.vargas@email.c
6	7	2024-03-07	Nicolás Vargas	iPad Air	749.99	La Plata	nicolas.vargas@email.c
11	12	2024-04-18	Sebastián Ruiz	MacBook Pro	2499.99	Mendoza	sebastian.ruiz@email.c
19	20	2024-08-01	Sebastián Ruiz	iPhone 15	1299.99	Mendoza	sebastian.ruiz@email.c
8	9	2024-03-22	Sofía Méndez	MacBook Air	1899.99	Salta	sofia.mendez@email.c
10	11	2024-04-10	Tomas Aguirre	iPad Pro	999.99	NaN	NaN

	id_venta	fecha	nombre_cliente	producto	monto_usd	ciudad	e
14	15	2024-05-28	Valentina Tores	Apple Watch	499.99	NaN	NaN
2	3	2024-01-20	Valentina Torres	iPad Pro	999.99	Rosario	valentina.torres@emai

Vemos que hay clientes con nombres muy similares a otros que aparecen en la base de clientes, pero que se diferencian por alguna cuestión tipográfica, como por ejemplo, la falta de alguna tilde o algún caracter.

3. Considerando que en `clientes_base.xlsx` los nombres de los clientes se encuentran exentos de errores ortográficos y tipográficos, ¿en qué porcentaje de los registros que conforman el dataset `ventas.xlsx` el nombre del cliente coincide con el de un cliente registrado?

```
# Serie de Pandas con valores booleanos
coincidencias = ventas['nombre_cliente'].isin(clientes['nombre_cliente'].unique())

# Total de registros coincidentes en el nombre
total_coincidencias = coincidencias.sum()

# Calculamos porcentaje sobre el número de registros de ventas
porcentaje = total_coincidencias*100/len(ventas)

print(f'En el {porcentaje} % de los registros de ventas coincide el nombre del cliente con
```

En el 70.0 % de los registros de ventas coincide el nombre del cliente con el de un client

4. Teniendo en cuenta lo observado en los ítems anteriores, utilice herramientas de *fuzzy joins* para realizar la unión de ambos datasets. ¿De qué ciudad es el cliente que más compras realizó en el local?

Tomamos como referencia el código proporcionado en la sección de [Ejemplos teórico-prácticos de apoyo](#), incluida dentro de este mismo apartado.

```

from rapidfuzz import process, fuzz, utils

# Definimos función de búsqueda
def buscar_coincidencia(nombre, opciones, umbral = 80):
    resultado = process.extractOne(
        nombre,
        opciones,
        scorer = fuzz.token_sort_ratio,
        processor = utils.default_process,
        score_cutoff = umbral
    )
    if resultado:
        return resultado[0], round(resultado[1],2)
    return "Ningún match", "No aplica"

# Buscamos coincidencias con un umbral de 80
ventas[['Nombre_matched', 'Score']] = ventas['nombre_cliente'].apply(
    lambda x: pd.Series(buscar_coincidencia(x, clientes['nombre_cliente'])))
)

# Exploramos resultados
print(ventas[['nombre_cliente', 'Nombre_matched', 'Score']])

```

	nombre_cliente	Nombre_matched	Score
0	Lucía Fernández	Lucía Fernández	100.00
1	Martin Gomez	Martín Gómez	83.33
2	Valentina Torres	Valentina Torres	100.00
3	Nicolás Vargas	Nicolás Vargas	100.00
4	Nicolas Vargas	Nicolás Vargas	92.86
5	Camila Herrera	Camila Herrera	100.00
6	Nicolás Vargas	Nicolás Vargas	100.00
7	Agustin Romero	Agustín Romero	92.86
8	Sofía Méndez	Sofía Méndez	100.00
9	Florencia Castro	Florencia Castro	100.00
10	Tomas Aguirre	Tomás Aguirre	92.31
11	Sebastián Ruiz	Sebastián Ruiz	100.00
12	Lucía Fernandez	Lucía Fernández	93.33
13	Nicolás Vargas	Nicolás Vargas	100.00
14	Valentina Tores	Valentina Torres	96.77
15	Camila Herrera	Camila Herrera	100.00
16	Florencia Castro	Florencia Castro	100.00
17	Agustín Romero	Agustín Romero	100.00
18	Nicolás Vargas	Nicolás Vargas	100.00
19	Sebastián Ruiz	Sebastián Ruiz	100.00

Se encontraron coincidencias aproximadas con *scores* altos en los 6 casos en los que la coincidencia no es exacta.

Realizamos la unión difusa de ambos DataFrames utilizando el campo `Nombre_matched` del dataset de ventas:

```

df_final = ventas.merge(clientes, left_on='Nombre_matched', right_on='nombre_cliente').drop(
df_final.head()

```

	id_venta	fecha	producto	monto_usd	nombre_cliente	ciudad	em
0	1	2024-01-05	iPhone 15	1299.99	Lucía Fernández	Buenos Aires	lucia.fernandez@email.c
1	2	2024-01-12	MacBook Air	1899.99	Martín Gómez	Córdoba	martin.gomez@email.co
2	3	2024-01-20	iPad Pro	999.99	Valentina Torres	Rosario	valentina.torres@email.c
3	4	2024-02-03	MacBook Pro	2499.99	Nicolás Vargas	La Plata	nicolas.vargas@email.co
4	5	2024-02-14	iPhone 15	1299.99	Nicolás Vargas	La Plata	nicolas.vargas@email.co

Respondemos a la pregunta por la ciudad del cliente que más compras realizó:

```
df_final['nombre_cliente'].value_counts().head()
```

```
nombre_cliente
Nicolás Vargas      5
Lucía Fernández     2
Valentina Torres    2
Camila Herrera      2
Agustín Romero      2
Name: count, dtype: int64
```

Vemos que el cliente que realizó más compras fue Nicolás Vargas. Extraemos directamente su nombre de la Serie anterior y buscamos la ciudad en la que vive:

```
cliente_max = df_final['nombre_cliente'].value_counts().idxmax()
ciudad_cliente_max = df_final[df_final['nombre_cliente'] == cliente_max].ciudad.iloc[0]

print(f'El cliente que más compras realizó es {cliente_max} y vive en {ciudad_cliente_max}')
```

```
El cliente que más compras realizó es Nicolás Vargas y vive en La Plata
```

Ejercicio N°3

La siguiente tabla representa cinco canciones con sus vectores de frecuencia de términos, contruidos a partir de las letras de cada una (vocabulario simplificado de 6 palabras):

	*"amor"	"noche"	"sol"	"lluvia"	"tiempo"	"vida"
cancion_1	8	1	0	0	2	5
cancion_2	6	0	0	1	3	4
cancion_3	0	7	3	5	1	0
cancion_4	1	6	2	4	0	0
cancion_5	5	0	8	0	1	3

1. ¿Qué medida propondría para evaluar la similaridad de las canciones, basada en el conjunto de términos que componen el vocabulario simplificado?
2. Calcule manualmente dicha medida considerando `cancion_1` y `cancion_2` y verifique el resultado utilizando herramientas de la librería `SciPy`.
3. Calcule la matriz completa de similaridades entre las cinco canciones y represéntela visualmente mediante un gráfico apropiado. ¿Qué pares de canciones resultan más similares entre sí?

Ejercicio N°4

Una consultora evaluó a un conjunto de candidatos en tres pruebas de selección, todas puntuadas en la misma escala de 0 a 100. Los datos se encuentran en el archivo

`candidatos.xlsx`.

1. Explore brevemente el conjunto de datos y realice un análisis exploratorio de la información que el mismo contiene.
 2. Se propone utilizar tres variables numéricas para calcular distancias entre candidatos. ¿Considera necesario estandarizarlas antes de calcular distancias? Justifique.
 3. Calcule las matrices de distancias euclídeas y Manhattan entre candidatos y visualícelas mediante un gráfico apropiado.
- Identifique pares de candidatos que sean muy similares o muy diferentes. ¿Observa grupos o patrones?
 - Según ambas métricas: ¿qué candidatos resultan más similares? ¿Qué candidatos resultan más diferentes? ¿Hay pares cuya similitud dependa de la métrica utilizada?